Find the Promos

Why I picked Problem 1

The brief states the bottleneck outright: an editor watches every episode and picks promo moments by hand, and a new show ships every week. So the cost repeats indefinitely, and unlike most things you'd point AI at, the output is checkable. A timestamp either exists in the screenplay or it doesn't. It's also the only one of the three problems that arrives with real data, which meant I could show the tool working on Kuku's own material rather than on test cases I'd written to suit myself.

I scored all three against the five lenses before writing any code (`docs/02_decision.md`), and Problem 1 led on the two I care most about: whether I could measure quality honestly, and whether the prototype would survive being re-run live on an input it hasn't seen. Problem 2 would have meant writing legal content with no legal background and no input pack, under a rule against inventing sections, which is a lot of ways to be confidently wrong. For Problem 3 I did find four real defects in this data, but none of them clears the brief's own bar of mattering more than Problems 1 and 2, and saying that seemed better than dressing up a QA chore as a business problem.

The obvious objection is that it's the obvious choice, so plenty of submissions will look similar. I decided to compete on method rather than topic.

Assumptions I had to make

A promo is a cut list, not a finished clip. The 20-60 second range and "hook inside 3 seconds" are hard limits in code, not preferences. Timestamps are relative to each episode's start, which I checked: all 50 begin at `00:00.00` and there's no series-level clock in the pack. Splicing within an episode is allowed, but only one promo crosses episodes, because stitching freely across 50 needs reliable character identity and this corpus doesn't have it. Runtimes also don't match the "1 to 2-minute" framing: median 118.9 seconds, but 12 of the 50 run past 150 and the longest is 200.8. I went with the data.

And there's no ground truth for "good promo", so the scoring weights are mine, hand-set, written down in one table. Nothing here is learned and I won't describe it that way.

How it works

The decision everything else follows from: **code owns time, the model owns judgment.** The model never sees a timestamp and never writes one. It reads beats as `[b07] JULIANA (desperate): "..."` and answers with beat IDs, then code looks each ID up in the parsed table. A hallucinated timestamp isn't unlikely in this design, it's impossible.

Parse (code). 50 files become 3,727 timed beats. The parser reconciles labels against timestamps against beats and refuses to continue if they disagree. It reports 22 defects it found in the source files. It also collapses 64 speaker labels into 58 characters, stripping an honorific only when the bare name turns up somewhere else in the corpus, which is why `LORD ASHCROFT` and `LADY ASHCROFT` stay two people.

Score (code). This is the part I got lucky on. The screenplays already label how almost every line is delivered: `(Cold, Commanding)`, `(Screaming, Desperate)`. 1,472 of 1,487 dialogue beats carry one, across 617 values, so the tool never has to guess where the drama is. A lexicon I wrote by hand (90% hit rate) turns those tags into an intensity curve, and a promo is a shape on that curve. From there: 18,141 legal windows on real beat boundaries, 3,376 thrown out for having nothing in the first three seconds, 171 shortlisted. 0.8 seconds, offline, free.

Select (model, one call per episode). Choose among the pre-validated windows and write the hook, cliffhanger and one line on why. It also tags a hook type from DramaBox's published taxonomy (moral outrage, romantic tension, empathy shock, power reversal) so an editor can spread types rather than posting four slaps in a row.

Splice (code). Two-piece promos inside one episode with 23-77 seconds of middle cut out, rejected unless something changes between the halves: either the speaker or the intensity. 41 of 91 candidates were two-piece.

Validate (code). Six pass/fail checks, then duplicate suppression on visual text between episodes no more than two apart. 37 dropped, ten of them real cross-episode repeats, including episodes 31 and 32, which both end on a branding iron inches from the heroine's eye.

Gate (model, one call per promo). A fresh instance sees only the clip, with no logline, episode number or surrounding beats, and has to say who this is, what's at stake, what question it's left with, and whether it would keep watching.

The tool never posts anything. It turns "watch 50 episodes and scrub" into "look over 47 ranked, timestamped candidates", and the editor's keep/reject clicks get logged, which is what eventually replaces my opinion with real preferences.

Cost at volume

One 50-episode show costs 127 API calls and about 200,000 tokens, measured rather than estimated. At a show a week that's 2,600 episodes a year, roughly 6,600 calls and 10.4M tokens. Free tier covers it outright; at Gemini Flash-Lite rates it's about $2.20 a year.

Tokens are not the scaling cost and I'd say so in a planning meeting. The real constraint on free tier is 8,000 tokens a minute, which stretches a show to roughly 20 minutes of wall clock and disappears the moment someone pays. What costs money is editor attention, which is why the expensive parts of this design exist.

How I know it's any good

Across all 92 promos, including rejected ones and the cross-episode promo, `timestamps_exist` passes 92/92 and so do the other five checks. The blind gate tested 54 promos, failed 7, left none untested, and by criterion scored ends-unresolved 1.00, character-identifiable 0.91, stakes-clear 0.76, would-keep-watching 0.65. Two runs give the identical fingerprint `c7b809ad92b976e7`. 42 of 50 episodes end up with a promo.

None of that answers the real question, which is whether the picks beat picking at random. So I tested it directly. Every pick goes up against a random legal window from the same episode: same length rules, same real beat boundaries, just not chosen. A blind judge sees two clips as plain text and never sees a score.

Against any legal 20-60 second window, which tests the whole system, the tool wins **75.8%** of the time after correcting for position, and 78.6% on a strictly counterbalanced subset where it won both ways on 10 of 14 pairs. Against a window that already passes the hook filter it wins **65.1%** (64.3% counterbalanced). Chance is 50%, n is 38 pairs, uncorrected p=0.0017.

One thing I nearly missed: the judge has a strong preference for whichever clip comes second, winning there 88.5% of the time against 41.7% shown first. So I re-judged as many pairs as the token budget allowed in the opposite order. Averaging the position-conditional rates and taking the counterbalanced subset give nearly the same answer, which is why I trust it.

The second comparison is the more useful number. Requiring a strong opening does most of the work; my tension scoring and the model's choice add a smaller but real edge on top. I'd rather report that than the flattering headline.

Splicing turned out to cost comprehension, which I hadn't expected to be measurable. Continuous promos pass the gate at 0.886, spliced ones at 0.800. I'd already charged a flat 0.10 penalty for the cut on instinct; this is what justifies it, and no spliced promo makes it near the top of the ranking.

One thing I removed on purpose: there's no hand-picked reference set as the headline measure. I built one, then demoted it. It asks "did the tool match one human's pick?", which isn't the job, and keeping one per show would recreate the manual work the tool is meant to remove. With six examples the per-episode spread was wider than the gap between approaches anyway. It survives as a sanity check (`docs/gold_set.json`, mean IoU 0.465). The version that scales is the editor's accept/reject log, which costs nothing extra because reviewing the shortlist is already the job.

Across two model families (`gpt-oss-120b` and `qwen3.8-27b`, 50 episodes each) mean agreement is 0.50 IoU, with identical picks on 17 of 50, no overlap at all on 8, and hook-type agreement of 0.48 on a four-way label. So a real part of this ranking is model taste rather than signal, which argues for keeping the editor in the loop. `qwen` also produced one validator failure where `gpt-oss-120b` produced none.

Where it falls over

It can't see. Shot quality, performance and pacing are invisible when your input is text, and the script tells me what gets said, not whether it lands. "Would keep watching" is the weakest gate criterion at 0.65, so clips are followable but not always compelling, and that gap is taste I can't measure. Eight episodes ship nothing, losing their best candidate to the gate or a duplicate.

Two more are my fault rather than the data's. The duplicate check nearly shipped dead: I set the threshold at 0.45 on dialogue text and it never fired once, and it took me a while to work out why. Across all 1,225 promo pairs, dialogue overlap peaks at 0.177, because the repetition in this show is visual and not verbal. And the alias map is still partly judgment. `ALEXANDER'S MOTHER`, `DOWAGER DUCHESS` and `MATRIARCH` are plausibly the same woman; I left them unmerged because I couldn't evidence it.

AI tools, and where they got it wrong

Claude for code and drafting. Groq-hosted `openai/gpt-oss-120b` as the pipeline model, `qwen/qwen3.8-27b` as the comparison. Five examples with before and after are in `docs/ai_mistakes_log.md`.

The one still in the output is in my top-ranked promo. `gpt-oss-120b` wrote its hook as "a royal guard slams Rupert to the ground, blood and dust flying". The beat it points at says he "firmly forces Rupert down into the stone dust", and the word blood appears in none of episode 44's beats. I checked every one. So the architecture holds where it matters, since the timestamps are verified, but the prose around them isn't, and an editor reading that line would cut to `00:14.00` expecting something that isn't there. Timestamps verified, descriptions not. A copy verifier that string-matches the nouns in a hook against the beats it covers is the next thing I'd build.

The other one was mine. When splicing started shipping, agreement with my reference spans jumped from 0.46 to 0.519, which looked like the feature had worked. It hadn't. My IoU measured a promo's outer envelope, so a two-piece promo got credit for the seconds the splice removes. Measuring the real segment union put it back to 0.465. Nothing errored, no test failed, and I only noticed because a marginal feature produced a 13% gain and that didn't smell right.